

Capítulo 7

React: componentes, hooks y formularios

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Presentar la teoría de interfaces React: estructura de aplicación, JSX, componentes, props, estado, eventos, listas, formularios, validación y hooks como base para consumir servicios backend.

Tabla de contenido

1. Panorama del capítulo	3
2. Del DOM imperativo al paradigma declarativo	3
3. JSX y renderizado	4
4. Virtual DOM y reconciliación	4
5. Componentes, props y composición	5
6. Estado y eventos	6
7. Hooks	7
8. Listas, formularios y validación	8
9. Estado compartido: levantamiento y Context	9
10. Estructura de una aplicación React	10
11. React 19: la evolución del modelo	10
12. Caso guía: formulario de reserva	11
13. Actividades sugeridas	11
14. Rúbrica breve de autoevaluación	12
15. Cierre	12

1. Panorama del capítulo

React organiza la interfaz en componentes. Cada componente representa una parte de la experiencia del usuario y puede recibir datos, manejar eventos y actualizar su estado. En una aplicación Full Stack, React no trabaja aislado: muestra datos que vienen del backend y envía acciones del usuario hacia la API. Este capítulo estudia los fundamentos conceptuales de ese modelo: por qué React describe la interfaz en lugar de manipularla, cómo se traduce esa descripción al navegador, y qué disciplina impone sobre los componentes, el estado, los eventos, los hooks y los formularios. Cierra con la organización de una aplicación completa y una mirada compacta a la evolución del modelo en React 19.

Idea central

Una interfaz React útil no es una colección de tarjetas decorativas. Es la materialización de una ecuación: la interfaz es una función del estado. Todo lo demás —JSX, hooks, formularios controlados, flujo unidireccional— existe para sostener esa ecuación sin excepciones.

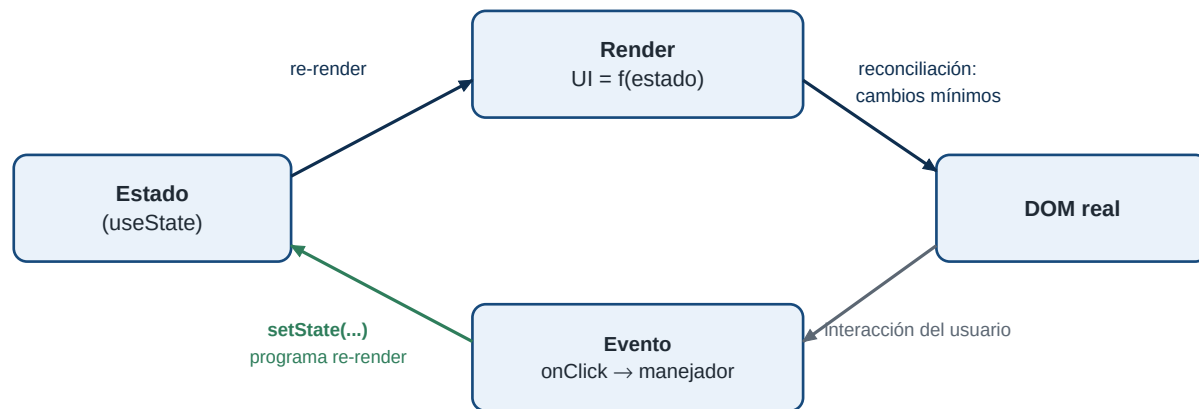
2. Del DOM imperativo al paradigma declarativo

En la programación imperativa de interfaces —el modelo del DOM clásico y de bibliotecas como jQuery— el programador manipula directamente los nodos del documento: busca un elemento, le cambia el texto, agrega o quita clases. Cada interacción exige escribir la transición exacta entre el estado visual anterior y el nuevo. El problema es de escala: si una pantalla puede encontrarse en N estados visuales, el número de transiciones posibles crece de forma combinatoria, y cada transición es código que alguien debe escribir, probar y mantener. Los errores típicos de las interfaces imperativas —un contador que no se actualiza, un mensaje de error que sobrevive a la corrección del campo— son transiciones olvidadas. Además, el código queda acoplado a la estructura concreta del HTML: un cambio de maquetación rompe la lógica aunque el comportamiento deseado no haya cambiado.

React invierte el modelo: el programador describe cómo se ve la interfaz para un estado dado — $UI = f(\text{estado})$ — y la librería se encarga de las transiciones. Cuando el estado cambia, React vuelve a ejecutar la función del componente, obtiene la nueva descripción y calcula por sí mismo qué modificar en el documento. Las transiciones dejan de ser código y pasan a ser una consecuencia automática de la diferencia entre dos descripciones. El costo es ceder el control fino del documento y expresar todo cambio visual como un cambio de datos; el beneficio es que el razonamiento deja de ser combinatorio: basta verificar que la descripción sea correcta para cada estado posible, sin importar desde cuál estado anterior se llegó. El mismo estado siempre produce la misma pantalla.

```
// Imperativo: describir la transición
document.querySelector('#total').textContent = carrito.length;

// Declarativo: describir el resultado para cualquier estado
function Resumen({ carrito }) {
  return <span>{carrito.length} productos</span>;
}
```



3. JSX y renderizado

JSX es la sintaxis con la que se escriben las descripciones de interfaz: una extensión de JavaScript que permite expresar estructuras similares a HTML dentro del código. Conviene precisar lo que no es: no es HTML (usa `className` en lugar de `class` y atributos camelCase como `onClick`), no es un motor de plantillas procesado en el servidor, y no reemplaza a CSS. Antes de ejecutarse, cada fragmento JSX se compila a llamadas de función de JavaScript que construyen elementos de React: objetos ligeros que describen el tipo de nodo, sus props y sus hijos. Un elemento no es un nodo del DOM; es el plano a partir del cual React construirá o actualizará nodos reales. Esta compilación explica las reglas de JSX: como una función devuelve un solo valor, el componente devuelve un único elemento raíz; como las llaves interpolan expresiones, dentro de ellas cabe cualquier expresión de JavaScript pero no sentencias.

```
// JSX que escribe el programador
const titulo = <h1 className="titulo">Hola, {nombre}</h1>;

// JavaScript al que compila (esquemático)
const titulo = jsx('h1', { className: 'titulo',
                           children: ['Hola, ', nombre] });
```

- Usar `className`, no `class`; los atributos siguen camelCase (`onClick`, `htmlFor`).
- Interpolan datos dinámicos entre llaves: `{usuario.nombre}`, `{total \* 1.07}`.
- Devolver un único elemento raíz; usar fragmentos (`<>...</>`) cuando no se desea un contenedor extra.
- Mantener la lógica compleja fuera del JSX: calcular antes, describir después.
- Usar claves (`key`) estables al renderizar listas (la razón se desarrolla en la sección 4).
- Renderizar condicionalmente con `{condicion && <Aviso />}` o el operador ternario.

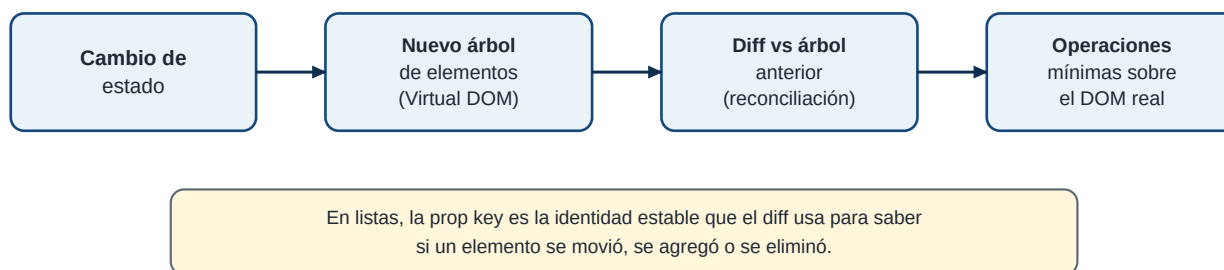
4. Virtual DOM y reconciliación

Si en cada cambio de estado React reconstruyera el DOM completo, el modelo declarativo sería impracticablemente lento: el DOM real es costoso de modificar, y reconstruirlo descartaría el estado del navegador (foco, posición de scroll, texto seleccionado). La solución es el Virtual DOM: una representación ligera en memoria del árbol de elementos. En cada render, React produce un árbol nuevo, lo compara con el

anterior —proceso llamado reconciliación o diffing— y calcula el conjunto mínimo de operaciones para llevar el DOM real del estado viejo al nuevo. La comparación usa heurísticas para ser lineal: si dos elementos tienen tipos distintos, React descarta el subárbol completo y lo reconstruye; si tienen el mismo tipo, conserva el nodo y actualiza solo los atributos que cambiaron. Renderizar es barato porque ocurre sobre objetos de JavaScript; tocar el DOM es caro, y por eso React lo hace lo menos posible.

Las listas plantean un caso especial. Cuando React compara dos versiones de una lista, necesita decidir qué elemento nuevo corresponde a cuál anterior. La prop `key` es la identidad estable de cada elemento: con ella React sabe si un elemento se movió, se agregó o se eliminó, y puede conservar su nodo y su estado. Sin `key` —o con el índice de posición como key— solo puede comparar por posición, lo que produce actualizaciones incorrectas o innecesarias cuando la lista se reordena, se filtra o recibe inserciones: el estado de un elemento (un campo escrito, una casilla marcada) queda adherido a la posición y no al dato. La key correcta proviene del dominio —el identificador del registro—; el índice solo es aceptable en listas estrictamente estáticas.

```
{reservas.map((r) => (  
  <ReservationCard key={r.id} reserva={r} />  
))}  
// key = identidad estable del dato, no la posición en el arreglo
```



5. Componentes, props y composición

Un componente es una función que recibe datos y devuelve una descripción de interfaz. Es la unidad de composición de React: las pantallas no se construyen como documentos monolíticos sino como árboles de componentes, donde cada uno tiene un propósito claro —formulario de login, tarjeta de producto, tabla de reservas, encabezado, botón de acción o vista de detalle—. Un componente bien diseñado puede reutilizarse sin depender de variables globales ocultas: todo lo que necesita le llega por props y todo lo que comunica sale por los manejadores que recibió. Esa autodescripción permite razonar sobre cada pieza en aislamiento.

Tipo de componente	Responsabilidad	Ejemplo
Layout	Organizar estructura común.	Header, Sidebar, PageContainer
Presentacional	Mostrar datos recibidos por props.	ReservationCard, StatusBadge
Formulario	Capturar y validar entrada.	LoginForm, ProductForm
Vista	Coordinar datos y componentes.	DashboardPage, DetailPage
Servicio/hook	Encapsular lógica de datos.	useAuth, useReservations

Las props son los datos de entrada de un componente y son de solo lectura. Esta inmutabilidad no es un capricho estilístico: garantiza que un componente sea predecible —con las mismas props produce la misma salida— y que los datos fluyan en una sola dirección, de padres a hijos. Si un hijo pudiera modificar sus props, cualquier componente podría alterar datos ajenos y rastrear un valor incorrecto exigiría revisar el árbol completo. Con flujo unidireccional, el origen de cada dato es único y conocido: el componente que lo posee como estado. La comunicación inversa no rompe la regla: el padre pasa una función como prop y el hijo la invoca cuando ocurre algo relevante. Los datos bajan; los eventos suben. Este patrón, junto con la prop `children`, constituye la composición: construir comportamientos complejos combinando piezas simples.

```
function FiltroEstado({ valor, onCambio }) {
  return (
    <select value={valor} onChange={(e) => onCambio(e.target.value)}>
      <option value="todas">Todas</option>
      <option value="activa">Activas</option>
    </select>
  );
}
// El dato (valor) baja por props; el evento (onCambio) sube al padre.
```

6. Estado y eventos

El estado representa información que cambia durante la interacción: valores de formulario, filtros, datos cargados, errores o sesión. En React tiene una semántica precisa: llamar a la función de actualización no muta la variable de inmediato, sino que registra el nuevo valor y programa un nuevo render. Durante un mismo render, el estado es una instantánea congelada; el valor nuevo solo existe en el render siguiente. Cada render es así una fotografía consistente donde todas las variables, props y manejadores ven los mismos valores. De este modelo se desprenden dos disciplinas: no mutar objetos ni arreglos directamente (`lista.push(item)` cambia el contenido pero no la referencia, y React no detecta el cambio), y no almacenar como estado lo que puede derivarse —un total, una lista filtrada—, porque todo dato duplicado puede desincronizarse.

```
const [count, setCount] = useState(0);

function manejarClic() {
  setCount(count + 1); // programa un re-render
  setCount(count + 1); // misma instantánea: count sigue valiendo 0
}
// Resultado: count pasa a 1, no a 2.
// Para encadenar: setCount((prev) => prev + 1)
```

Estado	Uso	Riesgo
Local	Formulario o componente específico.	Duplicarlo en varios componentes.
Compartido	Sesión, carrito, filtros globales.	Usar global para todo.
Remoto	Datos que vienen de la API.	No manejar carga/error.
Derivado	Cálculo a partir de otros datos.	Guardarlo como estado innecesario.

Los eventos capturan las acciones del usuario: clics, cambios de campos, envíos y navegación. React no entrega los eventos nativos del navegador, sino eventos sintéticos: envoltorios con interfaz uniforme entre navegadores que React administra de forma centralizada. El contrato es el del DOM (`preventDefault`, `target`), pero la suscripción se declara en JSX como una prop, manteniendo el comportamiento junto a la estructura que lo usa. Un detalle con consecuencias conceptuales: el manejador se asocia pasando la referencia de la función —`onClick={manejar}`—, no su invocación; `onClick={manejar()}` la ejecutaría durante el render, violando su pureza. El ciclo completo de interactividad queda cerrado: el usuario actúa, el manejador actualiza el estado, y el nuevo estado produce un nuevo render. La interfaz nunca se modifica directamente; siempre se modifica el estado del que la interfaz es función.

7. Hooks

7.1 Por qué existen y por qué tienen reglas

En el React original, solo los componentes de clase podían tener estado y ciclo de vida, y reutilizar esa lógica exigía patrones indirectos que oscurecían el árbol. Los hooks, incorporados en 2019, resolvieron ambos problemas: dan estado y efectos a los componentes de función —más simples de escribir, leer y probar— y permiten extraer lógica con estado en funciones reutilizables. Internamente, React identifica cada hook por el orden en que se llama durante el render: el primer `useState` corresponde a la primera celda de estado, el segundo a la segunda, y así sucesivamente. De ese mecanismo derivan las reglas de los hooks, que no son convenciones de estilo sino requisitos estructurales del modelo:

- Llamarlos solo en el nivel superior del componente: un hook dentro de un condicional o un bucle cambiaría el orden de llamadas entre renders y rompería la correspondencia entre llamadas y celdas de estado.
- Llamarlos solo desde componentes de función o desde otros hooks: fuera de un render no existe la estructura interna que les da significado.
- Nombrarlos con el prefijo `use`: permite a las herramientas de análisis estático verificar automáticamente las dos reglas anteriores.

7.2 useState

`useState` declara una celda de estado local y devuelve un par: el valor actual y la función para actualizarlo, con la semántica de instantánea estudiada en la sección 6. El criterio de granularidad importa: varios `useState` pequeños e independientes suelen ser preferibles a un objeto único, porque cada actualización reemplaza el valor completo de su celda y los estados independientes no deben obligarse a viajar juntos. Cuando varias piezas cambian siempre de forma conjunta, agruparlas vuelve a ser lo correcto. La regla práctica de ubicación: mantener el estado cerca de donde se necesita y evitar duplicarlo sin razón.

7.3 useEffect: sincronización con sistemas externos

El render de React debe ser puro: calcular la descripción de la interfaz y nada más. Todo lo que toca el mundo exterior —peticiones HTTP, suscripciones, temporizadores, el título del documento— es un efecto, y vive en `useEffect`, que se ejecuta después de que React actualizó el DOM. La forma correcta de pensar `useEffect` no es "código que corre al montar" sino sincronización: el efecto mantiene un sistema externo alineado con ciertos valores del componente, y el arreglo de dependencias declara cuáles son. El arreglo debe ser honesto —incluir todo valor que el efecto lee—: omitir dependencias produce efectos que trabajan con valores obsoletos; inventarlas produce ejecuciones innecesarias. Cuando el efecto crea algo que debe

deshacerse, devuelve una función de limpieza que React ejecuta antes de la siguiente sincronización y al desmontar.

Dependencias	Cuándo se ejecuta el efecto	Lectura conceptual
<code>useEffect(fn)</code>	Después de cada render.	Sincronizar con todo; raramente deseado.
<code>useEffect(fn, [])</code>	Una vez, tras el primer render.	Sincronización de montaje: no depende de valores.
<code>useEffect(fn, [a, b])</code>	Tras el primer render y cada vez que a o b cambien.	El sistema externo depende de a y b.

```
useEffect(() => {
  let activo = true;
  fetch(`/api/reservas?estado=${filtro}`)
    .then((r) => r.json())
    .then((datos) => { if (activo) setReservas(datos); });
  return () => { activo = false; }; // limpieza
}, [filtro]); // sincronizado con el filtro
```

Cuidado con useEffect

No todo pertenece a un efecto. Transformar datos para mostrarlos se hace durante el render; responder a una acción del usuario se hace en el manejador del evento. `useEffect` queda reservado para sincronizar con sistemas externos. Convertirlo en el lugar de toda la lógica de la aplicación produce cadenas de efectos difíciles de rastrear.

7.4 Hooks personalizados: reutilizar lógica, no interfaz

Un hook personalizado es una función cuyo nombre comienza con `use` y que combina hooks existentes. Su aporte es separar la lógica con estado de la presentación: la misma lógica de autenticación o de consumo de reservas puede usarse desde componentes visualmente distintos. Cada componente que llama al hook recibe su propio estado independiente —los hooks comparten lógica, no datos—. Un buen hook personalizado tiene nombre claro, responsabilidad única y un contrato pequeño: recibe parámetros simples y devuelve exactamente lo que el componente necesita, ocultando los detalles de sincronización.

```
function useReservas(filtro) {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => { /* fetch sincronizado con filtro */ }, [filtro]);
  return { datos, cargando, error };
}
// El componente consume el contrato sin conocer los detalles.
```

8. Listas, formularios y validación

Los formularios son el punto donde el usuario transforma intención en datos, y plantean una pregunta de diseño: ¿quién es el dueño del valor de un campo, el DOM o React? En un componente no controlado, el DOM guarda el valor y React lo lee cuando lo necesita (típicamente al enviar, mediante una ref): mínimo código, pero sin visibilidad del valor durante la escritura. En un componente controlado, el valor vive en el

estado de React y el campo solo lo refleja: cada tecla dispara `onChange`, que actualiza el estado, que re-renderiza el campo. El estado se convierte en la fuente única de verdad, y eso compra capacidades que el enfoque no controlado no puede ofrecer: validación inmediata mientras el usuario escribe, campos dependientes entre sí, transformación del valor al vuelo y botones de envío deshabilitados mientras los datos sean inválidos. Por eso es el enfoque por defecto en formularios con lógica; el no controlado queda para casos triviales.

```
const [email, setEmail] = useState('');

<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
// El estado es la fuente única de verdad; el input solo la refleja.
```

La validación no es una sola barrera sino una serie de capas con propósitos distintos. La primera es el HTML nativo (`required`, `type`, `min`): gratuita y accesible, pero limitada. La segunda es la validación en React: reglas de negocio expresadas sobre el estado, con mensajes específicos y momento de aparición controlado. La tercera es el backend, y es la única que cuenta como seguridad: todo lo que corre en el navegador puede ser alterado o evitado por el cliente. La validación del frontend debe coincidir con la del backend, pero no la reemplaza: existe para mejorar la experiencia, no para proteger los datos.

Capa	Propósito	Ejemplo
HTML nativo	Prevención inmediata y accesible.	required, type="email", min
React (estado)	Reglas de negocio y mensajes claros.	fecha no anterior a hoy; bloque disponible
Backend (API)	Integridad y seguridad reales.	400 con detalle de campos inválidos

- Definir campos requeridos y mensajes claros, redactados para el usuario y no para el programador.
- Mostrar errores cerca del campo correspondiente y en el momento oportuno.
- Deshabilitar el envío durante operaciones en curso para evitar duplicados.
- Preservar los valores escritos cuando ocurre un error: obligar a re-escribir es un defecto de diseño.
- Mostrar confirmación explícita si la operación fue exitosa.
- Enviar los datos en el formato exacto que el contrato de la API espera.

9. Estado compartido: levantamiento y Context

Tarde o temprano dos componentes necesitan ver el mismo dato: el filtro que un panel define y una tabla aplica. Como los datos solo fluyen hacia abajo, la solución canónica es levantar el estado: moverlo al ancestro común más cercano de los componentes que lo necesitan y repartirlo por props —el valor a quien lo muestra, la función de actualización a quien lo cambia—. El criterio general: mantener el estado tan abajo como sea posible y tan arriba como sea necesario. Levantarlo de más crea renders y acoplamientos innecesarios; levantarlo de menos obliga a duplicarlo.

Cuando un dato es necesario en ramas lejanas del árbol —la sesión del usuario autenticado, el tema visual, el idioma—, pasarlo por props a través de componentes intermedios que no lo usan (el llamado prop drilling)

ensucia los contratos de todos los niveles. Context es el mecanismo de React para ese caso: un proveedor coloca un valor en el árbol y cualquier descendiente lo lee directamente. Conceptualmente no es un almacén global sino un canal de difusión por subárbol: el valor sigue teniendo un dueño único (el componente proveedor) y el flujo sigue siendo unidireccional. Se justifica para datos transversales y de cambio poco frecuente; usarlo como sustituto de toda gestión de estado convierte cada cambio en un re-render de todos los consumidores y oculta las dependencias que las props hacían explícitas.

10. Estructura de una aplicación React

React no impone estructura de carpetas, pero dos esquemas dominan. La organización por tipo agrupa archivos según lo que son: todos los componentes juntos, todos los hooks juntos, todos los servicios juntos. Es simple al inicio y se degrada con el tamaño: trabajar en una funcionalidad obliga a saltar entre carpetas distantes. La organización por característica agrupa según el dominio: todo lo relacionado con reservas —componentes, hooks, servicios— vive junto. Escala mejor porque el código que cambia junto vive junto, y porque cada característica puede razonarse, probarse y eliminarse como unidad. Las piezas genuinamente genéricas (botones, modales, el cliente HTTP base) conservan carpetas comunes.

```
src/
├── features/
│   ├── auth/           # LoginForm, useAuth, authService
│   └── reservas/       # ReservationCard, useReservas, reservasService
├── components/        # piezas genéricas: Button, Modal, Layout
├── services/          # cliente HTTP base
└── App.jsx            # composición y rutas
```

Cualquiera que sea la organización, toda vista que consume datos remotos debe considerar al menos cuatro estados: cargando, vacío, con datos y error. No son adornos: son el contrato entre la interfaz y la incertidumbre de la red. Ignorarlos produce interfaces frágiles que muestran pantallas en blanco mientras llega la respuesta o colapsan ante el primer fallo. Planificar cómo se verá la pantalla antes, durante y después de la respuesta del backend es parte del diseño del componente, no un detalle posterior.

Estado	Qué ve el usuario	Ejemplo
Cargando	Indicador o skeleton.	Cargando reservas...
Vacío	Mensaje orientador.	No hay reservas registradas.
Con datos	Lista, tabla o detalle.	Reservas filtradas por estado.
Error	Mensaje accionable.	No se pudo cargar. Reintentar.

11. React 19: la evolución del modelo

Los fundamentos de este capítulo no cambian con las versiones: la interfaz sigue siendo función del estado, las props siguen siendo inmutables y los efectos siguen sincronizando con el exterior. Lo que React 19 ataca es el código asíncrono repetitivo que esos fundamentos dejaban en manos del programador: cargar datos, enviar formularios y reflejar operaciones pendientes obligaba a cada equipo a reinventar los mismos estados manuales de carga, error y éxito. React 19 incorpora ese patrón al núcleo de la librería.

API	Concepto que incorpora
use()	Leer recursos (una Promise o un Context) durante el render; al no ser un hook clásico, puede usarse dentro de condicionales.
Actions + useActionState	El ciclo completo de una mutación —envío, estado pendiente, error— gestionado por React en lugar de a mano.
useOptimistic	Interfaz optimista: mostrar el resultado esperado de inmediato y revertirlo solo si la operación falla.
ref como prop	Los componentes de función reciben ref como cualquier otra prop, sin forwardRef.
React Compiler	Memorización automática en tiempo de compilación; useMemo y useCallback dejan de ser responsabilidad del programador.

Material complementario

El desarrollo completo de React 19 —use() con Suspense, Actions y useActionState, useOptimistic, ref como prop y React Compiler, junto con su contraparte de backend en Hono— se encuentra en la guía complementaria del Módulo III sobre React 19 y Hono, disponible en los materiales del curso. Este capítulo aporta los fundamentos sobre los que esa guía se apoya.

12. Caso guía: formulario de reserva

El caso guía integra los conceptos del capítulo. Un formulario de reserva debe cargar los laboratorios disponibles (estado remoto sincronizado con `useEffect`` o equivalente), permitir seleccionar fecha y bloque (campos controlados con el estado como fuente única de verdad), validar por capas, enviar la solicitud al backend deshabilitando el envío mientras está en curso, mostrar los errores de choque de horario que el servidor reporte y actualizar la lista si la reserva se crea correctamente. Ninguna pieza exige técnicas avanzadas: el caso completo se resuelve con los fundamentos del capítulo, y la versión con Actions de React 19 reduce el código del envío sin alterar el diseño conceptual.

Elemento	Responsabilidad frontend	Contrato con backend
Select de laboratorio	Estado remoto: carga, vacío, datos, error.	GET /api/labs
Fecha y bloque	Campos controlados; validación inmediata.	POST /api/reservas
Estado de envío	Deshabilitar botón; evitar doble envío.	Respuesta 201 o error 400/409
Mensaje final	Confirmar creación o explicar corrección.	JSON con reserva o detalle de error
Lista actualizada	Re-sincronizar tras la mutación.	GET /api/reservas

13. Actividades sugeridas

- Descomponer una pantalla del proyecto en componentes, clasificándolos según la tabla de la sección 5.
- Definir las props (nombre, tipo, obligatoriedad) de tres componentes reutilizables.

- Justificar qué keys usaría en dos listas del proyecto y por qué el índice sería incorrecto.
- Diseñar los estados de carga, vacío, datos y error para una vista que consume la API.
- Convertir un formulario en controlado y documentar qué validaciones habilita ese cambio.
- Extraer la lógica de consumo de un recurso a un hook personalizado con contrato claro.
- Explicar qué validaciones se harán en frontend y cuáles en backend, y por qué las segundas son obligatorias.

14. Rúbrica breve de autoevaluación

Criterio	Logrado	Por mejorar
Paradigma	Explica UI = f(estado) y lo contrasta con el modelo imperativo.	Describe React como "HTML dentro de JavaScript".
Componentes	Props inmutables, flujo unidireccional, composición.	Componentes que mutan datos ajenos o dependen de globales.
Hooks	Justifica las reglas desde el orden de llamadas; useEffect como sincronización.	Usa useEffect como cajón de toda la lógica.
Formularios	Estado como fuente única de verdad; validación por capas.	Lee el DOM directamente; valida solo en el cliente.
Estados de UI	Toda vista remota contempla carga, vacío, datos y error.	Solo contempla el caso exitoso.

15. Cierre

React propone un contrato simple de enunciar y exigente de cumplir: describir la interfaz como función del estado y canalizar todo cambio a través de ese estado. De ese contrato se derivan, sin excepciones arbitrarias, los temas del capítulo: JSX como notación de descripciones, el Virtual DOM y la reconciliación como mecanismo que las vuelve eficientes, las keys como identidad, las props inmutables y el flujo unidireccional como garantía de predecibilidad, los hooks y sus reglas como consecuencia del modelo de render, y los formularios controlados como aplicación del principio de fuente única de verdad. Su valor en una aplicación Full Stack aparece cuando cada componente responde a una tarea real del usuario y se conecta correctamente con los datos del sistema: ese es el puente hacia el capítulo siguiente, dedicado a la integración con el backend y a las pruebas.